Document Information	
Filename	TP_F100U-00045_User_Guide_Device_Tree_Configuration
Status	Release
Revision Date	Jul 29, 2020
Version	02.00
Document ID	TP_F100U-00045

Standard Proprietary Terms

THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Table of contents

1 Introduction ... 3
1.1 Notational Conventions ... 3
2 Setup ... 3
2.1 Tools/Equipment Needed ... 3
2.2 Setup Instructions ... 3
2.3 Setup Troubleshooting ... 4
3 Features/Components ... 4
3.1 Device Tree Configuration ... 4
3.1.1 Overview ... 4
3.1.1.1 Platform Content Overview ... 6
3.1.1.2 Product Expectations ... 6
3.1.1.3 Feature for Development ... 6
3.1.1.4 Extending Platform Functionality ... 6
3.1.2 Setup ... 8
3.1.2.1 Tools/Equipment Needed ... 8
3.1.2.2 Setup Instructions ... 8
3.1.2.3 Setup Troubleshooting ... 9
3.1.3 Usage ... 9
3.1.3.1 Basic Instructions ... 9
3.1.3.2 Detailed Instructions ... 9
3.1.3.3 Testing/Error Reporting ... 11
4 Known Issues ... 11
5 FAQs ... 11
6 Troubleshooting/Tips and Tricks ... 11
7 References ... 11
7.1 Abbreviations/Acronyms/Definitions ... 11
7.2 Platform Documentation ... 12
7.3 Website links ... 12
8 Revision History ... 12

© Continental Automotive GmbH 2019
Confidential and Proprietary

1 Introduction

This document provides a description on how to modify the Linux Device Tree that is being used for Continental telematics devices.

The Linux device tree is defined as “The way to describe non-discoverable hardware”. This can be represented in several different formats.
The device tree data is typically created and maintained in a human readable format in .dts source files.

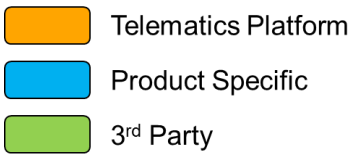
The document provides code snippet and examples based on sa515m chipset.

1.1 Notational Conventions

Ctrl+x	*A sequence with a ‘+’ indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*
…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	*A link to an external reference (e.g., website, document)*

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend



2 Setup

2.1 Tools/Equipment Needed

Any text editor like vim,nano,gedit,notepad++,etc.

2.2 Setup Instructions

As described in SDK User Guide(TP_F101U-00035) a dts is being provided on the kernel_amend.tar.gz file (Document section 3.2.1.2)

© Continental Automotive GmbH 2019
Confidential and Proprietary

The <product-name>.dts file can be used as basis for modifications.

Once the file has been modified it shall be copied to $CAS_WORKDIR/<linux-code>/arch/arm/boot/dts/qcom/

2.3 Setup Troubleshooting

Not Applicable

3 Features/Components

3.1 Device Tree Configuration

3.1.1 Overview

The device tree is a simple tree structure type data structure of nodes and properties. Properties are key-value pairs. The device tree is in byte code format which contains the information that is helpful to the kernel when booting up. Kernel parses the entry from the device tree and accordingly selects the platform hardware specific drivers.

There are below file format or associated file or file format with Device Tree.

Device tree source text file (*.dts) - source

Device tree binary blob (*.dtb) - object code

Device tree compiler (dtc) - Compiler

Device tree include text file (*.dtsi) - include source

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
/* this DTS is derived from sdxprairie-mtp.dts */

#include "sdxprairie-cas-pinout.dtsi"

/ {
    model = "<product-name>";
    compatible = "qcom,sdxprairie-mtp",
        "qcom,sdxprairie", "qcom,mtp";

    /* MSB in board-id is subtype:

            0x00:Chestnut
            0x01:<product-name>

          This value must match the subtype field in the corresponding CDT
          file.
       LSB is platform type (0x05 OEM as defined by QC)

            For information on the reserved field see kernel documentation:
                Documentation/devicetree/bindings/arm/msm/board-id.txt
    */
    qcom,board-id = <0x01000005 0x0>;

    aliases {
        /delete-property/i2c0; /* not used starting with B1 */
        /delete-property/i2c1;
        /delete-property/spi0;
    };
};

&board_gpio {
    led-backlight-pwr-on-gpio = <&tlmm 104 0x80>; /* output */
    ug5v-nad-disable-gpio     = <&tlmm 107 0x80>; /* output */
    nad-wdog-gpio             = <&tlmm  87 0x80>; /* output */
    wlan-nad-wake-gpio        = <&tlmm  81 0x80>; /* output */

    /delete-property/cdc-int1-n-gpio;  /* not used starting with B1 */
    /delete-property/cdc-reset-n-gpio; /* not used starting with B1 */
    /delete-property/vuc-boot-mode-gpio;
    /delete-property/vbatt-low-gpio;
};

/* ConMod has an audio codec but not managed by kernel */
&snd_934x {
    compatible = "qcom,sdx-asoc-snd-no-codec";
};

&tlv320aic3x_codec {
    status = "disable";
};
```

Model Property

Node Name

Property Name

Property Value

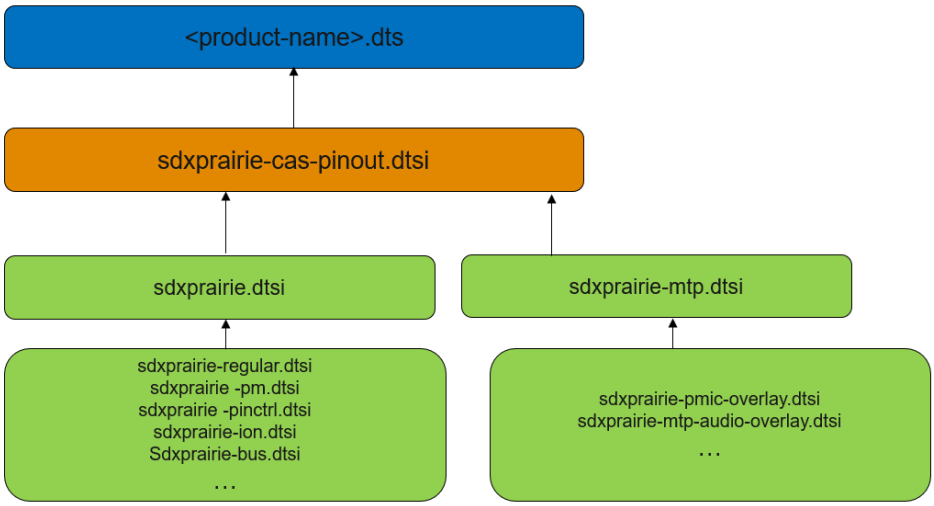
© Continental Automotive GmbH 2019
Confidential and Proprietary

If product team needs help in understanding on how to write a dts or dtsi file of their product specific peripheral or device, they can refer link provided in the section Website links.

3.1.1.1 Platform Content Overview

Telematics Platform creates the platform level, specific device tree including files from SOC specific level. Platform maintains the device tree configuration needed for the platform features or functionality.

Platform owned device tree files are just the include files which gets included in the hierarchy as depicted below and the attributes could be modified from the device tree source, which is part of platform reference and owned by each product group. This gives flexibility to product team to select or enable/disable certain hardware drivers.



3.1.1.2 Product Expectations

SDK Users shall only modify <product-name>.dts to add, extend, override the properties depending upon their hardware needs.

After changes to device tree, Linux needs to be recompiled and fresh images needs to be flashed to see the changes applied to device tree. Please refer SDK User Guide (TP_F101U-00035) to build Linux.

3.1.1.3 Feature for Development

The feature of separation of device tree form boot image (kernel) is only available with secure boot off and for development purpose only. The device tree customization can be done seamlessly without the recompilation of linux. The linux-dtsi.tar is provided inside the image folder for serving the purpose. The dts files along with the related header and dtsi files are provided in the tar to custom build the dtb. The dts files can be customized by user and dtb file can be generated from the script which is provided in the tar folder. User need to flash the generated custom dtb file to dtb_1 partition to override the device tree configuration. The concept is that the bootloader will load the device tree configuration from dtb_1 partition if flashed, by validating base memory, tag address and tag size.

3.1.1.4 Extending Platform Functionality

Adding a GPIO as output:

© Continental Automotive GmbH 2019
Confidential and Proprietary

1- Create a node named “&board_gpio” if is not existent already
2- Add the gpio ; as a property name inside the node that was just created
3- Add the description of the GPIO into the value of the property that we are adding.

```
&board_gpio {
        myD-gpio   = <&tlmm 101 0x80>; /* output */
        myC-gpio   = <&tlmm 102 0x80>; /* output */
        myB-gpio   = <&tlmm 103 0x80>; /* output */
        myA-gpio   = <&tlmm 104 0x80>; /* output */
```

In this example let´s say that we are adding “myA-gpio” which corresponds to the physical pin 104 and it is configured as output 0x80

Notes:
-Is not needed to map the property name to physical pin number.
-The physical pin number will depend on the device datasheet and the hardware schematic to map to the pin number needed according to your project needs. To get more details about pin number please refer FAQs
-104 was used and example.

Adding a GPIO as input:

To add a GPIO as input, in the property section where we are describing the GPIO the value shall be 0x00 to indicate that this gpio interface will be input

Let’s modify the previous example to describe myD-gpio as input

```
&board_gpio {
        myD-gpio   = <&tlmm 101 0x00>; /* input */
        myC-gpio   = <&tlmm 102 0x80>; /* output */
        myB-gpio   = <&tlmm 103 0x80>; /* output */
        myA-gpio   = <&tlmm 104 0x80>; /* output */

};
```

Adding a GPIO from the PMIC:

To add a GPIO from the PMIC module, in the property section where we are describing the GPIO the value “tlmm” which indicates the source for the GPIO will need to be replaced to indicate that this GPIO interface come from the PMIC.

Let’s modify the previous example to describe myD-gpio and myC-gpio and as input and output from the PMIC

Notes:

© Continental Automotive GmbH 2019
Confidential and Proprietary

-The location property can vary the naming from device to device. This example is based on sa515
-The BIAS value CANNOT be modified from the device tree.

```
&board_gpio {
        myD-gpio   = <&pmxprairie_gpios 5 0x00>; /* input */
        myC-gpio   = <&pmxprairie_gpios 6 0x80>; /* output */
        myB-gpio   = <&tlmm 103 0x80>; /* output */
        myA-gpio   = <&tlmm 104 0x80>; /* output */

};
```

Accessing GPIO through Linux file system in target:

If the changes to the device tree are applied as specified in the device tree you will find the board_gpio into the Linux file system, inside this directory you will find the gpio interfaces that were added

```
[root@sa515m board_gpio]#pwd
pwd
/sys/firmware/devicetree/base/board_gpio
[root@sa515m board_gpio]#ls
ls
myA-gpio        myB-gpio     myC-gpio      myD-gpio
```

3.1.2 Setup

3.1.2.1 Tools/Equipment Needed

Not Applicable

3.1.2.2 Setup Instructions

Inside the image folder (e,g. 4K/2K) linux-dtsi.tar is found. This tar is provided to customize the device tree and make dtb out of it without recompiling the linux. The following steps are needed to customize the device tree and flashing it into dtb_1 partition. The steps are for linux environment and not for windows.

1. Extract the tar file.
   ```
   tar -xvf linux-dtsi.tar
   ```
2. The dts files can be found inside path “linux-dtsi\arch\arm64\boot\dts\qcom”
3. Customize the dts file.
4. Generate customized dtb image by executing the script file “generated-dtb” found in extracted tar folder
   ```
   ./generated-dtb
   ```
5. Generated dtb files and dtb image (in case of multiple dts files) can be found in path “linux-dtsi\arch\arm64\boot\dts” and can be flashed to dtb_1 partition.
6. Copy the generated dtb file or dtb.img to the desired folder for flashing.

© Continental Automotive GmbH 2019
Confidential and Proprietary

7. Send the board to bootloader

```
[root@sa515m ~]#sys_reboot bootloader
```

8. Flash the dtb_1 partition with dtb (eg. <Name of dtb file>.dtb) or image file (dtb.img).

```
fastboot flash dtb_1 <Name of dtb file>.dtb
```

OR

```
fastboot flash dtb_1 dtb.img
```

9. Reboot board

```
fastboot reboot
```

3.1.2.3 Setup Troubleshooting

Not Applicable

3.1.3 Usage

3.1.3.1 Basic Instructions

Not Applicable

3.1.3.2 Detailed Instructions

Platform HAL API

Register GPIO Change Notification

API/Type Description	Doxygen Path
API to register for GPIO Change Notification	Modules |-> Hardware Abstraction Layer (HAL) |-> General Purpose Input Output |-> **tpHAL_LowLevel_registerGpioChangeNotification()**
Callback prototype to receive notifications.	Modules |-> Hardware Abstraction Layer (HAL) |-> General Purpose Input Output |-> **tpHAL_gpioChangeNotification_t**

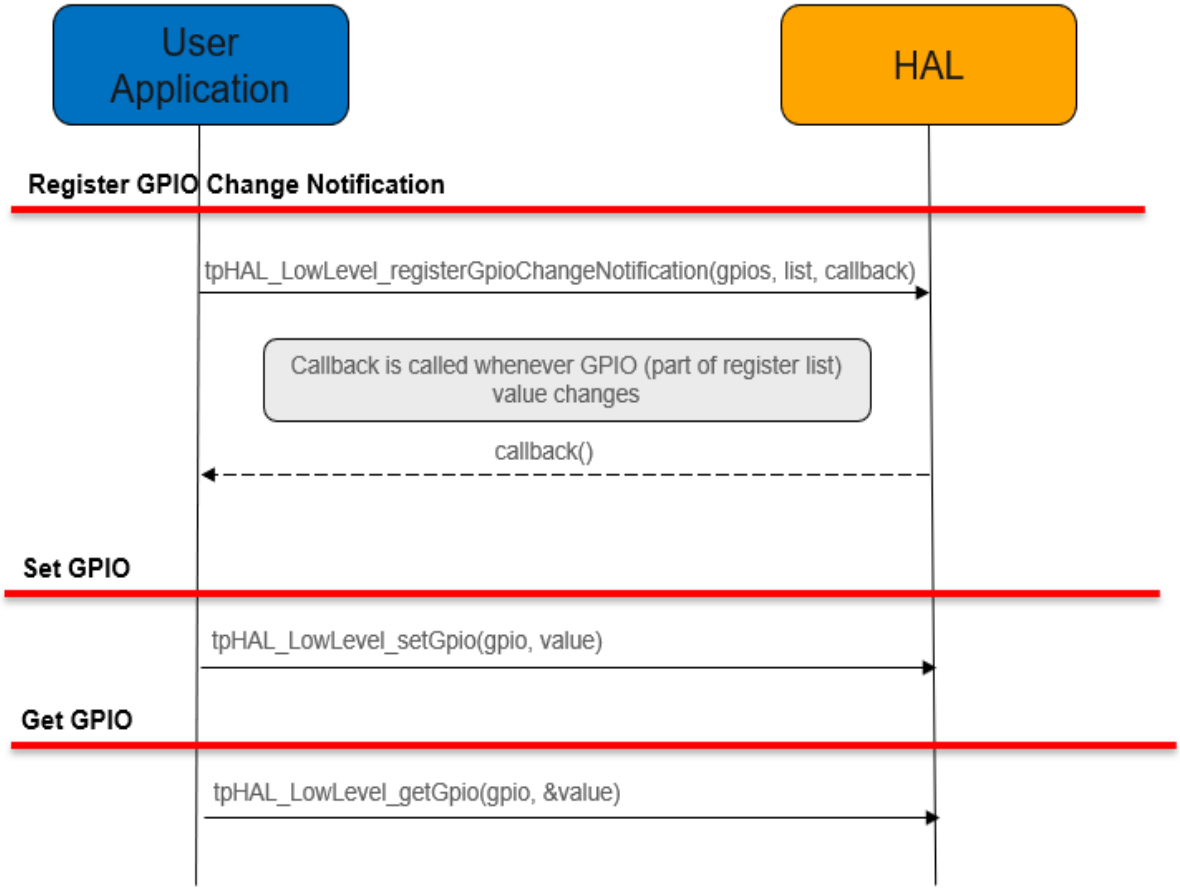
Set GPIO

API/Type Description	Doxygen Path
API to set GPIO	Modules |-> Hardware Abstraction Layer (HAL) |-> General Purpose Input Output |-> **tpHAL_LowLevel_setGpio()**

Get GPIO

API/Type Description	Doxygen Path
API to get GPIO	Modules |-> Hardware Abstraction Layer (HAL) |-> General Purpose Input Output |-> **tpHAL_LowLevel_getGpio()**

© Continental Automotive GmbH 2019
Confidential and Proprietary



Verify that GPIO can be accessed by tool “cas-gpio-utility”:

The following command can be sent to access the new GPIO that was added to the device tree via gpio utility, the command for this utility are the following:

-To read the current state of a gpio “get-gpio”

-To write a new state to the gpio “set-gpio”

Example on how to read, update and verify the new state of a given gpio

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
[root@sa515m data]#cas-gpio-utility --get-gpio bt-en-gpio
cas-gpio-utility --get-gpio bt-en-gpio
0
[root@sa515m data]#cas-gpio-utility --set-gpio bt-en-gpio 1
cas-gpio-utility --set-gpio bt-en-gpio 1
[root@sa515m data]#cas-gpio-utility --get-gpio bt-en-gpio
cas-gpio-utility --get-gpio bt-en-gpio
1
[root@sa515m data]#
```

Verify GPIO by sysfs

Product team can also export the GPIO using the Linux standard sysfs method. Linux opensource reference documentation is provided in the **Website links**

3.1.3.3 Testing/Error Reporting

Not Applicable

4 Known Issues

Not Applicable

5 FAQs

How to find the pin number?

Product team can find the pin number from the Hardware NAD Specification document.

How to determine if pin is from PMIC?

PMIC GPIO pin follows the convention PM_GPIO_XX, so if product team finds the name of GPIO starting with PM_GPIO it means that pin is from PMIC

6 Troubleshooting/Tips and Tricks

Not Applicable

7 References

7.1 Abbreviations/Acronyms/Definitions

PMIC: Power Management Integrated Circuit

© Continental Automotive GmbH 2019
Confidential and Proprietary

BIAS: Reference DC voltage operating point.
GPIO: General Purpose Input Output

7.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary
TP_F101U-00035: User Guide SDK

7.3 Website links

https://elinux.org/Device_Tree_What_It_Is
https://elinux.org/Device_Tree_Reference
Device trees I: Are we having fun yet?
Device trees II: The harder parts
http://xillybus.com/tutorials/device-tree-zynq-1
https://www.kernel.org/doc/Documentation/gpio/sysfs.txt

8 Revision History

Version	Date	Description
01.00	Feb 24, 2020	Initial release
02.00	Jul 29, 2020	Add separation of device tree from boot image

© Continental Automotive GmbH 2019
Confidential and Proprietary